FD E-COMMERCE ANALYTICS DASHBOARD

REVISED PROJECT SYNOPSIS OF INTERDISCIPLINARY MINOR PROJECT

Submitted by: Jagjot Singh Passi (2446571)
Course: Bachelor of Computer Applications (BCA)
Institute: Punjab College of Technical Education, Ludhiana
Course Instructor: Ms. Sarbjeet Kaur
Report revision: October 2026 — reflects the implemented Python–MongoDB–Streamlit solution

1. INTRODUCTION

The FD E-Commerce Analytics Dashboard is an interactive web-based business intelligence system that analyzes sales trends, customer purchase behavior, and marketing campaign performance for an FD-themed e-commerce retail example. It unifies sales, customer, product and advertising records using six linked datasets and then presents understandable key performance indicators, interactive charts and practical business suggestions. The completed prototype is built using **Python, Streamlit, Pandas, Plotly and MongoDB**. SQLite is available solely as a labeled offline demonstration database.

The included records are a transparent, reproducible simulated demonstration. Actual performance measurement requires properly sourced and permissioned store exports, which can be imported through validated CSV/Excel tools.

2. RATIONALE

Small sellers often store orders, customers, product information and marketing results in separate spreadsheets. Manually comparing these sources creates delays and data errors. A centralized dashboard helps store owners investigate category performance, returning customers, marketing spend and attributed conversions without repeating spreadsheet calculations. Import checks and provenance labels are required to ensure that the dashboard does not turn incorrect records into convincing-looking figures.

3. OBJECTIVES

1. Create an interactive integrated e-commerce analytics dashboard accessible in a browser.
2. Persist product, customer, order and marketing records in MongoDB.
3. Validate and import customer, order, product and campaign records from Excel/CSV sources.
4. Provide sales trends, category and region comparisons, and delivered-order KPIs.
5. Analyze buying patterns using RFM customer segments, repeat purchase rates and monthly cohorts.
6. Calculate campaign-level and channel-level ROAS, CPA and a bounded contribution ROI proxy.
7. Present dynamic filters, data exports, database backup, audit logs and decision-support guidance.
8. Demonstrate correctness through automated unit and integration tests.

4. LITERATURE REVIEW

Business intelligence tools such as Power BI, Tableau and open-source dashboard systems show how consolidated records and visual KPIs can support management. **Streamlit** reduces separate frontend engineering for Python-based analysis. **Plotly** supports interactive graphs, while **Pandas** enables reproducible data transformation and group aggregations. **MongoDB** provides persistent data collections. Customer segmentation is informed by RFM theory, and campaign effectiveness is reported using precisely defined attributed sales and marketing cost metrics. Official Streamlit, MongoDB, Pandas, Plotly and scikit-learn documentation were used as implementation references.

5. FEASIBILITY STUDY

Technical feasibility: A laptop with Python, a browser and a MongoDB service can run the project. A local SQLite fallback removes dependency on a running MongoDB server for offline evaluations. All dependencies appear in a requirements file.

Economic feasibility: Local development uses freely available tools. Optional cloud databases and hosting may have platform-specific limitations or charges.

Operational feasibility: An analyst can filter the dashboard, inspect metric definitions and download calculated reports. Administrators can import complete linked datasets and back them up.

Data and privacy feasibility: The bundle uses simulated data. Authorized real exports must be anonymized or otherwise handled under applicable permissions and data-protection rules.

6. METHODOLOGY / PLANNING OF WORK

Phase A — Requirements: identify business goals and the six necessary related record categories.

Phase B — Design: create entity relationship, logical architecture and data-flow diagrams. Define required fields, customer/transaction references, order status rules and documented marketing attribution.

Phase C — Dataset preparation: generate a coherent synthetic demonstration and document field semantics. Support direct import of six-tab Excel workbooks or six CSV datasets. Reject invalid foreign keys, order totals, dates, campaign assignments and negative values.

Phase D — Backend and storage: implement Python repository adapters; write MongoDB document collections with an active batch marker; add a SQLite transactional offline mode and load/audit metadata.

Phase E — Analysis and frontend: implement sales KPIs, RFM and cohort computations, marketing ROAS/CPA/contribution ROI, product/region trends and rule-based recommendations. Render them in eight Streamlit pages with Plotly visuals.

Phase F — Testing and demonstration: run Python automated tests; show dynamic filters, database collections, input validation, CSV/Excel exports, backup and responsible data disclosure.

7. FACILITIES REQUIRED FOR PROPOSED WORK

Hardware: Computer with 8 GB RAM recommended, internet connection for package installation or cloud MongoDB.
Software: Python 3.11+, Streamlit, Pandas, NumPy, Plotly, pymongo, scikit-learn, MongoDB Community/Atlas, MongoDB Compass (optional), browser, Git, VS Code.
Data: Six validated CSV or Excel tables; realistic simulated demo is included. Actual store datasets must be provided under authorization.

8. EXPECTED OUTCOMES / DEMONSTRABLE RESULTS

The application demonstrates delivered-order revenue trends, leading categories/products/regions, purchasing behavior by customer cohorts and RFM segments, channel-wise advertising attribution with ROAS and CPA, product contribution estimates, and data-driven operational suggestions. Analysts can change date/category/region filters and export summaries. An administrator can replace the complete dataset, download an Excel template, back up records and review change history. The project is a demonstrable analytics prototype; it is not a deployed live-store transaction integration or audited financial reporting platform.

9. REFERENCES

1. Streamlit: https://docs.streamlit.io/
2. MongoDB: https://www.mongodb.com/docs/
3. Pandas: https://pandas.pydata.org/docs/
4. Plotly Python: https://plotly.com/python/

5. scikit-learn: https://scikit-learn.org/stable/
6. Python: https://docs.python.org/3/
7. UCI Online Retail dataset: https://archive.ics.uci.edu/dataset/352/online+retail

Correction from original synopsis: This version replaces the proposed MERN implementation description with the Python/MongoDB/Streamlit architecture actually delivered in the project package. Data are synthetic until genuine records are collected and verified.